Bounding the Probability That an LLM Agent Exceeds Its Budget: Run-Level Risk Control with Learn-then-Test

Robat Das
Independent Researcher
ORCID: 0009-0006-7792-8351

DRAFT — November 5, 2026

Abstract

LLM agents are billed per token, and the cost of a run is not known until it ends: the number of steps, the model chosen at each step, and the length of every response are random. Operators therefore face a choice between reserving for the expected cost, which overruns the budget an uncontrolled fraction of the time, and reserving for the worst case, which is safe but so conservative that a capable model is almost never affordable. We ask for something in between: a controller whose probability of exceeding a run budget B is at most a user-chosen ε. We first show why the natural approach fails. Bounding each step's output length with a conformal or binomial quantile and splitting ε across steps is valid but, at realistic calibration sizes, collapses to worst-case reservation, because the union bound charges every step for a tail event that almost never causes a run-level overrun. We then calibrate the controller at the level of whole runs. A one-parameter family of reservation policies is tested in a fixed sequence with exact binomial p-values (Learn-then-Test), which yields $\Pr\big(\Pr(\text{spend} > B) \le \varepsilon\big) \ge 1 - \delta$ from i.i.d. runs alone, with no assumption on per-step output distributions. In simulation the calibrated controller keeps overruns below ε (in the no-truncation mode) while assigning 31–45% of steps to the capable model, where per-step bounds and worst-case reservation assign none; across 500 independent calibrations the guarantee held in 499 (well-specified simulator) and 495 (misspecified simulator) cases at $\delta = 0.05$. The method is implemented in the open-source library Baar-Core. On 2,100 real gpt-4o-mini and gpt-4o agent runs on HotpotQA, replayed exactly under each policy, worst-case reservation finished no run; the calibrated policy matched expected-cost reservation's completion at looser budgets, traded some completion for fewer overruns at the tightest one, and its guarantee held in at least 499 of 500 calibrations in every setting.

1 Introduction

An LLM agent alternates between model calls and tool calls until it produces an answer. Each model call costs $p_m^{\text{in}} n_t + p_m^{\text{out}} y_t$, where m is the model, n_t the prompt length (which grows as the conversation grows) and y_t the output length. Only n_t is known before the call. Measurement studies report that repeated runs of the same agentic task can differ by up to 30× in total tokens, and that agents predict their own consumption poorly [3, 10]. Unbounded consumption is listed among the top risks for LLM applications [13], and practitioners have catalogued dozens of budget-overrun incidents in agent frameworks [9].

A budget enforcer must decide, before each call, whether the call is affordable. To do so it must plan for an unknown output length. Two plans are common. Reserving the *expected* output is permissive but gives no control over how often the budget is exceeded. Reserving the *maximum*

output the provider allows never exceeds the budget, but a capable model's maximum output is often priced close to the whole budget, so the enforcer rarely admits it. Neither plan lets the operator say how much risk they will accept.

We study the following guarantee. Given a run budget B, a tolerance ε and a confidence level $1-\delta$, deploy a controller π such that, with probability at least $1-\delta$ over calibration,

$$R(\pi) \;=\; \Pr_{\text{task, sampling}}\big(\text{run cost under } \pi > B\big) \;\le\; \varepsilon. \tag{1}$$

The controller is adaptive: it chooses models and admits or refuses calls based on the money already spent, so per-step observations are not exchangeable and standard per-call conformal guarantees do not compose directly into (1).

Contributions.

1. We formalize run-level budget risk for adaptive agent controllers in two operating modes: one where responses are never cut off and the risk is overrun, and one where a `max_tokens` backstop prevents overrun and the risk is truncation (Section 3).
2. We show that per-step quantile bounds combined with a union bound or risk spending are valid but degenerate to worst-case reservation at practical calibration sizes, and explain why (Section 4).
3. We calibrate a one-parameter policy family at the run level with Learn-then-Test [1], giving the guarantee (1) from i.i.d. runs with no distributional assumption on output lengths (Section 5). The statistical tool is not new; the contribution is the formulation and the evidence that run-level calibration is what makes the guarantee usable.
4. We evaluate in a simulator with heavy-tailed outputs, growing context and adaptive routing, including a misspecified variant, and validate the guarantee over 500 independent calibrations (Section 6). We then replay 2,100 real agent runs exactly under each policy (Section 7).
5. We release the method as a reservation mode in the open-source Baar-Core library.

2 Related work

Cost-aware routing and cascades. FrugalGPT [6] and RouteLLM [11] trade cost against quality by choosing among models, evaluated by average cost. C3PO [14] optimizes cascades under a probabilistic constraint on the cost of a *single query*. Our constraint is on the total cost of a multi-step *run* under an adaptive controller.

Budget enforcement for agents. Khan [9] catalogues 63 production overrun incidents and enforces a dollar cap with an affine-typed Rust library, reporting static over-reservation of 4–6×. Green SARC [5] places enforcement gates in the agent loop and calibrates its per-action cost forecasts with split-conformal prediction. These systems make the cap hard by reserving conservatively, or calibrate individual actions; to our knowledge none bounds the probability that an entire adaptive run exceeds its budget while letting the operator trade that probability for capability.

Distribution-free risk control. Conformal prediction and its risk-control extensions [2, 4] bound the expected loss of a calibrated predictor. Learn-then-Test [1] recasts the choice of a hyperparameter as multiple hypothesis testing and controls arbitrary, non-monotone risks. Anytime-valid variants handle sequentially arriving data [8], and ToolChain-CRC applies trajectory-level conformal risk control to agent correctness under drift [12]. DAPRO [7] allocates a compute budget dynamically across multi-turn LLM evaluations with distribution-free coverage guarantees, satisfying the budget in expectation rather than with high probability. We apply Learn-then-Test to budget risk, where the relevant loss is defined over whole runs.

3 Problem setup

A task X is drawn i.i.d. from a distribution $\mathcal{D}$. A run proceeds in steps $t = 1, 2, \ldots$. Before step t the controller knows the money spent so far s_{t-1}, the prompt length n_t, and the history. It picks a model m_t and a step type k_t (tool call or final answer), or refuses, which ends the run without an answer. After the call it observes the output length y_t and pays $c_t = p^{\text{in}}_{m_t} n_t + p^{\text{out}}_{m_t} y_t$.

Reservation policies. The controller admits a call if $s_{t-1} + p^{\text{in}}_m n_t + p^{\text{out}}_m u(m,k) \le B$, where $u(m,k)$ is a planned output length. To leave room to finish, a tool call is admitted only if a subsequent final answer on the cheap model also fits. The plan u defines the policy:

- **estimate**: u is the mean output length;
- **preflight**: $u = Y^{\max}_m$, the provider's maximum output;
- **calibrated** (ours): $u = \hat{q}_\beta(m,k)$, the empirical $(1-\beta)$ quantile of output length for stratum (m,k), estimated on held-out logs. β is a single scalar chosen by calibration.

Two modes. In **Mode A** responses are never cut off, and the run loss is $L_A = \mathbf{1}\{\sum_t c_t > B\}$. In **Mode B** each call carries `max_tokens` equal to the largest output that still fits the remaining budget. Overrun becomes impossible and the loss is truncation, $L_B = \mathbf{1}\{\text{some response was cut off}\}$. Mode A is the default in Baar-Core because silent truncation can corrupt downstream results without any visible error. The risk of policy π is $R(\pi) = \mathbb{E}[L(\pi)]$.

Preflight has $R = 0$ in both modes: it never overruns, and in Mode B its `max_tokens` is never binding.

4 Why per-step bounds collapse

The natural route to (1) bounds each step. Choose levels α_t with $\sum_t \alpha_t \le \varepsilon$ (a union bound, or an online "risk-spending" schedule that spends a fraction of the remaining ε only when a step is tight), and plan u_t at an upper confidence bound on the $(1-\alpha_t)$ quantile. If every $y_t \le u_t$, the run cannot overrun, so the overrun probability is at most $\sum_t \alpha_t$ plus the calibration failure probability.

This is valid, but expensive in two ways.

Proposition 1. *Let $Y_1, \ldots, Y_n$ be i.i.d. and $Y_{(j)}$ their order statistics. The smallest order statistic that is a valid $(1-\alpha)$ upper quantile bound with confidence $1-\delta'$ is $Y_{(j)}$ with the least j such that $\Pr(\mathrm{Bin}(n, 1-\alpha) \ge j) \le \delta'$. No order statistic qualifies, and the bound must fall back to $Y^{\max}$, unless*

$$n \;\ge\; \frac{\log(1/\delta')}{-\log(1-\alpha)} \;\approx\; \frac{\log(1/\delta')}{\alpha}.$$

Proof. $Y_{(j)}$ lies below the $(1-\alpha)$ quantile only if at least j samples do, which has probability at most $\Pr(\mathrm{Bin}(n, 1-\alpha) \geq j)$. Even $j = n$ requires $(1-\alpha)^n \leq \delta'$. □

With $\varepsilon = 0.05$ spread over ten steps ($\alpha = 0.005$), and δ' divided further across strata and quantile levels so that all bounds hold simultaneously (e.g. $\delta' = 0.05/160$ for 4 strata and 40 levels), the proposition requires about 1,600 logged outputs per stratum merely to beat $Y^{\max}$. Risk spending makes later steps stricter still, because the remaining budget of risk shrinks geometrically.

The deeper problem is that the union bound charges every step for its own tail. A single long response almost never causes a run-level overrun, because most runs finish well under budget. The per-step bound for the capable model, at a tail level of a fraction of a percent, is priced near the entire budget, so the capable model is never admitted. Table 1 shows exactly this: the per-step policy achieves zero overruns and zero use of the capable model, indistinguishable from preflight. Run-level calibration avoids the union bound altogether.

5 Run-level calibration with Learn-then-Test

Fix in advance a grid $\beta_1 < \beta_2 < \cdots < \beta_J$ ordered from conservative to aggressive, and the per-stratum quantile tables $\hat{q}_\beta$, estimated on data disjoint from calibration. For each β_j the calibrated policy π_{β_j} is fully specified, so its run-level risk $R_j = R(\pi_{\beta_j})$ is a fixed number. We test the null hypotheses $H_j : R_j > \varepsilon$ in order.

Procedure. For $j = 1, 2, \ldots$: execute n fresh runs on i.i.d. tasks under π_{β_j} with budget B, count failures F_j, and compute the exact binomial p-value

$$p_j \;=\; \Pr\big(\mathrm{Bin}(n, \varepsilon) \leq F_j\big).$$

If $p_j \leq \delta$, reject H_j and continue; otherwise stop. Deploy $\hat{\beta} = \beta_{\hat{\jmath}}$, the last rejected level, or preflight if H_1 is not rejected.

Theorem 1. *If calibration runs are i.i.d. draws of (task, sampling randomness) from the deployment distribution, then* $\Pr\big(R(\pi_{\hat{\beta}}) > \varepsilon\big) \leq \delta$*, where the probability is over the calibration runs. This holds for any output-length distributions, any dependence between steps, and with no monotonicity of* R_j *in* j*.*

Proof. If $R_j \leq \varepsilon$ for all j, the deployed policy (any π_{β_j}, or preflight with risk 0) has risk at most ε. Otherwise let j^* be the smallest index with $R_{j^*} > \varepsilon$; it is deterministic. The deployed policy has risk above ε only if $\hat{\jmath} \geq j^*$, which requires H_{j^*} to be rejected. Under H_{j^*}, $F_{j^*} \sim \mathrm{Bin}(n, R_{j^*})$ with $R_{j^*} > \varepsilon$, which is stochastically larger than $\mathrm{Bin}(n, \varepsilon)$; hence $\Pr(p_{j^*} \leq \delta) \leq \delta$. □

This is the fixed-sequence instance of Learn-then-Test [1]. Three properties matter for budgets. First, the guarantee is about whole runs, so it accounts for adaptive routing, growing context and correlated step lengths without modelling them. Second, the per-stratum quantile tables only shape the policy family; if they are wrong, the procedure selects a more conservative β rather than losing validity. Third, the same procedure controls truncation in Mode B by changing the loss.

Cost of calibration. Each tested level requires n executed runs, because routing depends on β and runs cannot be replayed counterfactually. The total is $(\hat{\jmath}+1)n$ runs. In Mode A some calibration runs overrun by design; this spend should be accounted for when budgeting the calibration itself.

Table 1: Well-specified simulator, $B = \$0.02$, $\delta = 0.05$, 20,000 test runs. Failure is overrun in Mode A and truncation in Mode B. Estimate and preflight do not depend on ε.

Mode	Policy	$\hat{\beta}$	Failure	Big	Done
A	estimate	–	4.69%	44.6%	88.8%
A	preflight	–	0	0	100%
A	per-step, $\varepsilon \in \{.01, .05, .10\}$	–	0	0	100%
A	LTT, $\varepsilon = 0.01$	0.1	0.37%	31.3%	94.0%
A	LTT, $\varepsilon = 0.05$	0.2	2.63%	41.4%	90.4%
A	LTT, $\varepsilon = 0.10$	0.5	6.46%	45.2%	88.4%
B	estimate	–	13.55%	44.6%	88.8%
B	LTT, $\varepsilon = 0.01$	0.01	0	0	100%
B	LTT, $\varepsilon = 0.05$	0.05	3.08%	24.4%	96.3%
B	LTT, $\varepsilon = 0.10$	0.1	5.26%	31.3%	94.0%

6 Simulation study

Simulator. Two models with list prices of \$0.15/\$0.60 (small) and \$2.50/\$10.00 (capable) per million input/output tokens, and maximum outputs of 4,096 and 8,192 tokens. Output lengths are lognormal per stratum (model × step type), with the capable model and final answers longer. Each run starts with a 600-token prompt; each tool step appends the output and an observation of 50–800 tokens. The number of steps the task needs is geometric with mean 6, capped at 30. The controller prefers the capable model while more than half the budget remains, downgrades to the small model when the capable one does not fit, and wraps up early on the small model if necessary. The run budget is $B = \$0.02$. In the **misspecified** scenario every task has a hidden difficulty that scales all its output lengths by e^d, $d \sim \mathcal{N}(0, 0.6^2)$, so steps within a run are correlated and the per-stratum model is wrong.

Settings. Quantile tables use 2,000 held-out samples per stratum. Learn-then-Test uses $\beta \in \{0.001, 0.002, 0.005, 0.01, 0.02, 0.05, 0.1, 0.2, 0.3, 0.5\}$, $n = 1{,}000$ calibration runs per level and $\delta = 0.05$. The per-step baseline uses exact binomial quantile bounds with Bonferroni correction across strata and quantile levels and lazy risk spending. Each policy is evaluated on 20,000 fresh test runs. "Big" is the fraction of executed steps on the capable model, our proxy for how much capability the budget buys; "done" is the fraction of runs that reach a final answer rather than being refused.

Results. Table 1 shows the well-specified case. Estimate mode overruns 4.7% of runs regardless of the operator's tolerance, and in Mode B it truncates 13.6% of runs. Preflight and the per-step policy never fail but never use the capable model. Learn-then-Test meets every target and, at $\varepsilon = 0.05$, spends 41% of steps on the capable model while overrunning 2.6% of runs, close to estimate mode's capability at about half its overrun rate and with a certificate. At $\varepsilon = 0.01$ in Mode B no aggressive level could be certified and the procedure correctly fell back to near-preflight behaviour. The misspecified scenario gives the same picture (Table 2): estimate mode overruns 5.3%, and Learn-then-Test meets every target with 29–46% capable-model steps.

Is the guarantee real? Theorem 1 is a statement about the calibration draw, so we repeat the whole procedure 500 times per scenario with independent calibration runs ($\varepsilon = \delta = 0.05$, Mode A)

Table 2: Misspecified simulator (hidden per-task difficulty), Mode A.

Policy	$\hat{\beta}$	Overrun	Big	Done
estimate	–	5.25%	45.2%	85.9%
preflight / per-step	–	0	0	100%
LTT, $\varepsilon = 0.01$	0.1	0.50%	29.1%	92.4%
LTT, $\varepsilon = 0.05$	0.2	2.65%	41.0%	87.6%
LTT, $\varepsilon = 0.10$	0.5	7.05%	45.9%	85.7%

and measure each deployed policy's overrun rate on 4,000 fresh runs. The deployed policy violated ε in 1 of 500 calibrations in the well-specified scenario (0.2%; 95% Clopper–Pearson interval 0.0–1.1%) and 5 of 500 in the misspecified scenario (1.0%; 0.3–2.3%), both well below $\delta = 5\%$. These counts are slightly inflated by Monte Carlo error in the 4,000-run estimate; the largest measured overrun rate was 5.17%. Mean overrun was 3.0% and 2.8%, and capable-model share was 42.4% and 40.4%.

The price of control. Learn-then-Test completes fewer runs than preflight (88–94% versus 100%), because a more aggressive plan admits capable-model steps that later leave too little budget to finish, and the controller refuses. Completion is not part of the controlled loss here. A loss of "overrun *or* incomplete" is equally valid under Theorem 1 and would trade capability for completion; we leave its study to future work.

7 Real-model study

Setup. We ran a ReAct-style agent [16] on the HotpotQA distractor validation set [15]. The agent sees the titles of the ten context paragraphs, may call a `lookup(title)` tool to read one, and stops with a final answer or after eight steps (temperature 0.7, answers scored by token F1). We collected 1,500 runs with gpt-4o-mini and 600 with gpt-4o on the same shuffled tasks, logging every call's prompt tokens, Baar-Core's pre-call prompt estimate, output tokens and cost. Total spend was $1.85, itself hard-capped by Baar-Core in preflight mode.

Exact replay. A controller that only admits or refuses calls does not change what the agent does before it refuses. Each logged run can therefore be replayed under any reservation policy and budget, giving the exact outcome that policy would have had on that task and sampling draw. Reservations use the pre-call prompt estimate that was logged before each call, exactly as the controller saw it. We set the run budget B to the 50th, 75th or 90th percentile of unconstrained run cost, so that it binds for 50%, 25% or 10% of runs. For Learn-then-Test, a random quarter of each model's runs fits the quantile tables; the rest form a population whose true risk under every β is computed exactly. We draw calibration sets of half the population size i.i.d. from it, run the procedure ($\varepsilon = \delta = 0.05$), and repeat 500 times.

What real traces look like. Outputs are short and light-tailed: the 99th percentile is 50–67 tokens for both models and both step types, and the maximum observed is 181. The provider maximum of 16,384 tokens is 250–330 times the 99th percentile. Cost is dominated by input tokens, as reported for coding agents [3]; still, the most expensive run cost 8–11 times the median. The logged pre-call prompt estimate *under*-counts in 94% of calls, by a median of 29 tokens (9%; 37

Table 3: Real-trace replay on HotpotQA ($\varepsilon = \delta = 0.05$). B is set at a percentile of unconstrained run cost. "Done" is the fraction of runs that finish; F1 counts refused runs as 0. LTT shows the most frequently selected $\hat{\beta}$ and its exact risk; "viol." is the number of the 500 calibrations whose selected policy had risk above ε. Preflight finished no run at any of these budgets.

		estimate			Learn-then-Test				
Model	B	Overrun	Done	F1	$\hat{\beta}$	Overrun	Done	F1	viol.
4o-mini	p50	5.3%	54.7%	.328	0.1	2.8%	51.0%	.308	1/500
4o-mini	p75	1.9%	77.0%	.489	0.5	2.8%	78.0%	.496	0/500
4o-mini	p90	0.4%	89.7%	.584	0.5	0.6%	90.0%	.586	0/500
4o	p50	4.9%	54.7%	.317	0.05	1.6%	44.4%	.258	1/500
4o	p75	1.8%	75.1%	.458	0.3	1.6%	74.4%	.454	0/500
4o	p90	0.4%	88.4%	.554	0.5	1.1%	89.3%	.559	0/500

tokens on the first step), because Baar-Core's token counter could not take the tool schema, which providers bill as prompt tokens. Recounting with the schema matched the billed prompt to within one token. This study exposed the bug, and it is fixed in Baar-Core; we keep the original estimate in the replay because it is what the controller actually used. This matters below.

Results. Table 3 reports the replay. Four findings stand out.

1. **Worst-case reservation is unusable.** At every budget, the reserved cost of a 16,384-token output exceeds the budget, so preflight finishes no run. This is the failure the method exists to fix.

2. **The guarantee holds on real runs.** Across six settings and 3,000 calibrations, at most one calibration per setting selected a policy with risk above ε, against an allowed $\delta = 5\%$.

3. **Outcomes match estimate mode; the gain is the certificate.** At the 75th and 90th percentile budgets, estimate mode happened to overrun less than ε, and the certified policy reaches the same completion and F1. At the tight median budget estimate mode sits at 4.9–5.3%, at the edge of the tolerance; Learn-then-Test cuts overruns to 1.6–2.8% at a cost of 4–10 points of completion. The difference is that the certified policy's risk is guaranteed in advance, while estimate mode's depends on a workload the operator cannot see beforehand.

4. **Residual risk has more than one source.** Even at $\beta = 0.001$, gpt-4o overruns 0.4–0.7% of runs at the tighter budgets. Replaying with the true prompt counts separates two causes. At the median budget all of these overruns remain, so they come from outputs longer than any in the 150-run quantile table (whose extreme quantile is 67 tokens, against an observed maximum of 157). At the 75th percentile budget, correcting the prompt count removes two thirds of them (0.7% to 0.2%). Run-level calibration absorbs both errors because it measures realized overruns; a per-step output bound covers neither.

Two caveats. On this task the capable model was not more accurate (F1 0.61 versus 0.63 unconstrained), so we evaluate each model separately rather than the cheap/capable routing of Section 6; routing changes the trajectory and cannot be replayed exactly. And HotpotQA outputs are short, so the gap between preflight and calibrated reservation is mostly about the provider maximum; agents that write code or long answers would have heavier tails.

8 Implementation

The calibrated plan is a reservation mode in Baar-Core, an open-source Python library that enforces LLM budgets before each request is sent. Baar-Core estimates the cost of a call locally, atomically reserves it against a budget, reconciles with the provider-reported cost afterwards, and routes between a cheap and a capable model with downgrading. It already offers the estimate and preflight plans as `estimate` and `preflight` output modes, and Mode B as an opt-in `truncate` mode.

The method adds a `calibrated` mode and a dependency-free module `baar.calibration`. `QuantileTable` builds per-stratum output quantiles from logged calls; `ltt_select` runs the fixed-sequence procedure of Section 5 given a function that executes n runs at a level β and returns the failure count; exact binomial p-values are computed in log space. The router then reserves the calibrated quantile:

```
table = QuantileTable.from_records(logged_calls)
res = ltt_select(betas, run_agent_batch, n=1000, eps=0.05, delta=0.05)
router = BAARRouter(budget=0.10, on_output_overrun="calibrated",
                    output_quantiles=table, calibration_beta=res)
```

The mode never sets `max_tokens`, and it falls back to preflight when no level is certified or a stratum has too few samples. One gap remains: the router does not yet know whether a step is a tool call or a final answer, so it pools both step types per model; per-type strata are available through the lower-level `plan_output` call. The code is available at commit `c989f34b4498` of the repository below and will ship in the next Baar-Core release.

9 Limitations

- **Exchangeability.** The guarantee assumes deployment tasks come from the calibration distribution. Price changes, model updates or a new task mix require recalibration; anytime-valid monitoring [8] is a natural complement.
- **Calibration cost.** Each tested level costs n real runs at budget B. With $n = 1{,}000$ and up to ten levels this is affordable for cheap agents and expensive for long-horizon ones. Smaller n forces more conservative choices.
- **Coarse policy family.** One scalar β shared by all strata is simple to certify but not optimal; richer families need multiple-testing corrections that cost power.
- **Completion.** As discussed, completion falls as capability rises unless it is included in the loss.
- **Scope of the real study.** One task family, two models from one provider, and single-model runs; the routing results are simulated. On real traces the certified policy matched, rather than beat, expected-cost reservation in outcomes. Exact replay covers admit/refuse controllers only.

10 Conclusion

Operators of LLM agents can now state a budget tolerance instead of choosing between unbounded risk and worst-case reservation. Per-step tail bounds are the obvious way to get such a guarantee,

but they spend the risk tolerance on tail events that rarely matter. Calibrating whole runs with Learn-then-Test gives a finite-sample guarantee from i.i.d. runs alone and, in simulation, recovers most of the capability that worst-case reservation gives up.

Reproducibility. Simulator, calibration code, the real-model harness and all raw results are in the `research/` directory of `https://github.com/orvi2014/Baar-Core`, at commit `c989f34b4498`.

AI usage. Generative AI tools (Claude Opus and Claude Sonnet, Anthropic, through Claude Code) were used to write simulation and analysis code and to draft and edit this paper. The author chose the research question and method, checked the proofs, reviewed all code and text, and is responsible for the content. All reported numbers are produced by the released scripts.

References

[1] Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. Learn then test: Calibrating predictive algorithms to achieve risk control, 2021. arXiv:2110.01052.

[2] Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. Conformal risk control, 2022. arXiv:2208.02814.

[3] Longju Bai, Zhemin Huang, Xingyao Wang, Jiao Sun, Rada Mihalcea, Erik Brynjolfsson, Alex Pentland, and Jiaxin Pei. How do AI agents spend your money? analyzing and predicting token consumption in agentic coding tasks, 2026. arXiv:2604.22750.

[4] Stephen Bates, Anastasios Angelopoulos, Lihua Lei, Jitendra Malik, and Michael I. Jordan. Distribution-free, risk-controlling prediction sets, 2021. arXiv:2101.02703.

[5] Gaston Besanson. Green SARC: Predictive cost and carbon governance for agentic AI systems, 2026. arXiv:2606.15954.

[6] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance, 2023. arXiv:2305.05176.

[7] Shai Feldman and Yaniv Romano. Spend where it counts: Dynamic budget allocation for multi-turn LLM evaluation, 2026. arXiv:2605.06605.

[8] Bror Hultberg, Dave Zachariah, and Antônio H. Ribeiro. Anytime-valid conformal risk control, 2026. arXiv:2602.04364.

[9] Sajjad Khan. Token budgets: An empirical catalog of 63 LLM-agent budget-overrun incidents, with an affine-typed Rust mitigation as a case study, 2026. arXiv:2606.04056.

[10] Yuxiang Lin, Zihan Wang, Mengyang Liu, Yuxuan Shan, Longju Bai, Junyao Zhang, Xing Jin, Boshan Chen, Jinyan Su, Xingyao Wang, Jiaxin Pei, and Manling Li. BAGEN: Are LLM agents budget-aware?, 2026. arXiv:2606.00198.

[11] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data, 2024. arXiv:2406.18665.

[12] Jeffery Opoku and David Banahene. ToolChain-CRC: Conformal risk control for agentic AI under retrieval and tool-use drift, 2026. arXiv:2606.18467.

[13] OWASP Foundation. OWASP top 10 for LLM applications 2025: LLM10 unbounded consumption, 2025. URL `https://genai.owasp.org/llmrisk/llm102025-unbounded-consumption/`.

[14] Antonios Valkanas, Soumyasundar Pal, Pavel Rumiantsev, Yingxue Zhang, and Mark Coates. C3PO: Optimized large language model cascades with probabilistic cost constraints for reasoning. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2511.07396.

[15] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering, 2018. arXiv:1809.09600.

[16] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models, 2022. arXiv:2210.03629.